

System Information

Basic System Information

- `uname` - Displays basic system information about the kernel
- `uname -a` - Shows all system information including kernel version, hostname, and architecture
- `uname -r` - Displays the kernel release version
- `uname -m` - Shows the machine hardware architecture (e.g., x86_64)
- `hostname` - Displays the system's host name
- `hostname -I` - Shows the system's IP address
- `hostname -f` - Displays fully qualified domain name
- `hostnamectl` - Shows detailed host and operating system information
- `cat /etc/os-release` - Displays information about the operating system
- `lsb_release -a` - Shows distribution-specific information

Hardware Information

- `lscpu` - Displays CPU architecture information
- `cat /proc/cpuinfo` - Shows detailed CPU information for each core
- `lspci` - Lists all PCI devices
- `lspci -v` - Shows detailed information about PCI devices
- `lsusb` - Lists all USB devices
- `lshw` - Lists hardware configuration (requires root)
- `dmidecode` - Shows hardware info from the BIOS (requires root)
- `free` - Shows memory usage
- `free -h` - Shows memory usage in human-readable format
- `free -m` - Shows memory usage in megabytes

Disk Information

- `df` - Shows disk space usage
- `df -h` - Shows disk space usage in human-readable format
- `df -i` - Shows inode usage
- `du` - Shows directory space usage
- `du -sh /path/to/directory` - Shows total size of a directory in human-readable format
- `lsblk` - Lists block devices with their mount points
- `fdisk -l` - Shows disk partitions (requires root)
- `blkid` - Shows block device attributes (UUID, type)
- `mount` - Shows mounted file systems
- `mount | column -t` - Shows mounted file systems in a table format

Network Information

- `ip a` - Shows network interfaces and IP addresses
- `ip addr show` - Displays detailed network interface information
- `ifconfig` - Shows network interfaces (deprecated, use `ip a`)
- `ip route` - Shows routing table
- `netstat -r` - Shows routing table (alternative)
- `ss -tulnp` - Shows listening ports and their processes

- `netstat -tulnp` - Shows listening ports (deprecated, use ss)
- `ping hostname` - Tests connectivity to a host
- `host example.com` - Performs DNS lookup
- `dig example.com` - Advanced DNS lookup
- `nslookup example.com` - Alternative DNS lookup tool

Date and Time

- `date` - Shows current date and time
- `date +%Y-%m-%d` - Shows date in YYYY-MM-DD format
- `date +%T` - Shows time in HH:MM:SS format
- `date -u` - Shows date and time in UTC
- `uptime` - Shows system uptime and load averages
- `timedatectl` - Controls system time and date
- `cal` - Shows calendar for current month
- `cal -y` - Shows calendar for current year
- `cal 3 2023` - Shows calendar for March 2023

User Information

- `whoami` - Shows current username
- `id` - Shows user and group IDs
- `w` - Shows who is logged in and what they're doing
- `who` - Shows who is logged in
- `last` - Shows last logins
- `users` - Lists currently logged-in users
- `finger username` - Shows user information

File System Navigation

- `pwd` - Print working directory (shows current directory)
- `ls` - List files and directories in current directory
- `ls -l` - List files with detailed information (long format)
- `ls -a` - List all files including hidden ones (starting with .)
- `ls -la` - Combine long format and show all files
- `ls -lh` - List files with human-readable sizes
- `ls -lt` - List files sorted by modification time
- `ls -ls` - List files sorted by size
- `ls -R` - List files recursively including subdirectories
- `ls -ld */` - List only directories
- `cd directory` - Change to specified directory
- `cd` - Change to home directory
- `cd ~` - Change to home directory (alternative)
- `cd ..` - Move up one directory level
- `cd -` - Return to previous directory
- `cd /` - Change to root directory
- `find . -name "filename"` - Find files by name in current directory and subdirectories
- `locate filename` - Find files using the locate database
- `updatedb` - Update the locate database (usually run by cron)

- `which command` - Show full path of a command
- `whereis command` - Show locations of command's binary, source, and man page

File Creation & Viewing

Creating Files

- `touch filename` - Create an empty file or update timestamp of existing file
- `touch file1 file2 file3` - Create multiple files at once
- `echo "text" > file.txt` - Create a file with content
- `echo "more text" >> file.txt` - Append text to an existing file
- `cat > file.txt` - Create a file and write content (Ctrl+D to save)
- `printf "formatted text\n" > file.txt` - Create a file with formatted text
- `nano filename` - Create/edit a file with the nano editor
- `vi filename` - Create/edit a file with the vi editor
- `vim filename` - Create/edit a file with the improved vi editor
- `gedit filename` - Create/edit a file with the GNOME editor (GUI)
- `emacs filename` - Create/edit a file with the Emacs editor

Viewing File Content

- `cat filename` - Display file contents
- `cat -n filename` - Display file contents with line numbers
- `more filename` - Display file contents one screen at a time
- `less filename` - View file contents with advanced navigation
- `head filename` - Display first 10 lines of a file
- `head -n 20 filename` - Display first 20 lines of a file
- `tail filename` - Display last 10 lines of a file
- `tail -n 15 filename` - Display last 15 lines of a file
- `tail -f filename` - Display last lines and follow file updates in real-time
- `view filename` - View file in read-only mode (using vi)
- `nl filename` - Display file with line numbers
- `strings filename` - Display printable characters in binary files
- `hexdump -C filename` - Display file contents in hexadecimal format

Directory Operations

Creating Directories

- `mkdir dirname` - Create a directory
- `mkdir -p parent/child/grandchild` - Create parent directories as needed
- `mkdir dir1 dir2 dir3` - Create multiple directories at once
- `mkdir -m 755 dirname` - Create a directory with specified permissions

Removing Directories

- `rmdir dirname` - Remove an empty directory
- `rm -r dirname` - Remove a directory and its contents recursively
- `rm -rf dirname` - Force remove a directory and its contents without prompting
- `rmdir -p parent/child/grandchild` - Remove empty directories and their parents

File Operations

Copying Files

- `cp source destination` - Copy a file
- `cp -r source_dir destination_dir` - Copy directories recursively
- `cp -a source destination` - Copy files while preserving attributes
- `cp -u source destination` - Copy only when source is newer than destination
- `cp -i source destination` - Interactive copy (prompts before overwrite)
- `cp -v source destination` - Verbose copy (shows files being copied)
- `cp file1 file2 directory/` - Copy multiple files to a directory

Moving and Renaming Files

- `mv source destination` - Move/rename a file or directory
- `mv -i source destination` - Interactive move (prompts before overwrite)
- `mv -u source destination` - Move only when source is newer than destination
- `mv -v source destination` - Verbose move (shows files being moved)
- `mv file1 file2 directory/` - Move multiple files to a directory
- `rename 's/old/new/' files` - Rename files using regular expressions

Removing Files

- `rm filename` - Remove a file
- `rm -i filename` - Interactive removal (prompts before removal)
- `rm -f filename` - Force removal without prompting
- `rm file1 file2 file3` - Remove multiple files
- `shred -u filename` - Securely delete a file by overwriting its contents

Linking Files

- `ln -s target linkname` - Create a symbolic link to a file or directory
- `ln target linkname` - Create a hard link to a file
- `readlink linkname` - Show where a symbolic link points to
- `unlink linkname` - Remove a symbolic link

File Comparison & Searching

Comparing Files

- `diff file1 file2` - Show differences between two files
- `diff -u file1 file2` - Show differences in unified format
- `diff -y file1 file2` - Show differences side by side
- `sdiff file1 file2` - Side-by-side merge of file differences
- `cmp file1 file2` - Compare two files byte by byte
- `md5sum filename` - Generate MD5 checksum of a file
- `sha1sum filename` - Generate SHA1 checksum of a file
- `sha256sum filename` - Generate SHA256 checksum of a file

Finding Files

- `find /path -name "pattern"` - Find files by name

Example: `find /home -name "*.txt"` This searches for all .txt files in the /home directory and its subdirectories.

- `find /path -type f -size +100M` - Find files larger than 100MB

Example: `find /var -type f -size +100M` Searches for regular files larger than 100MB in the /var directory. The `-type f` specifies regular files, and `+100M` means larger than 100 megabytes.

- `find /path -mtime -7` - Find files modified in the last 7 days

Example: `find /home/user -mtime -7 -mtime -7` means files modified less than 7 days ago.

- `find /path -user username` - Find files owned by a specific user

- `find /path -perm 777` - Find files with specific permissions

- `find /path -exec command {} \;` - Execute a command on found files

Example: `find /tmp -name "*.log" -exec rm {} \;` This finds all .log files in /tmp and removes them. The `{}` is replaced with each found file, and `\;` terminates the `-exec` command.

- `locate pattern` - Find files using the locate database (faster than find)

File Permissions & Ownership

Viewing Permissions

- `ls -l filename` - Display file permissions and ownership
- `stat filename` - Display detailed file status including permissions
- `getfacl filename` - Display file access control lists

Changing Permissions

- `chmod permissions filename` - Change file permissions

Example: `chmod 755 script.sh` Sets permissions to `rxr-xr-x` (owner can read/write/execute, group and others can read/execute)

In numeric mode:

- 7 = `rxw` (read, write, execute)
- 5 = `rx` (read, execute)
- 4 = `r--` (read only)
- `chmod u+x filename` - Add execute permission for the owner
- `chmod g+w filename` - Add write permission for the group
- `chmod o-r filename` - Remove read permission for others
- `chmod a+x filename` - Add execute permission for all users
- `chmod -R 755 directory` - Change permissions recursively

Changing Ownership

- `chown user:group filename` - Change file owner and group
- `chown user filename` - Change file owner only
- `chown :group filename` - Change file group only
- `chown -R user:group directory` - Change ownership recursively
- `chgrp group filename` - Change file group

Special Permissions

- `chmod u+s filename` - Set SUID bit (run as owner)
- `chmod g+s filename` - Set SGID bit (run as group or inherit directory group)
- `chmod +t directory` - Set sticky bit (restrict deletion)
- `umask` - Display or set the default permission mask
- `umask 022` - Set the default permission mask (subtract from 666/777)

Text Processing Commands

Basic Text Processing

- `cat` - Concatenate and display files

Example: `cat file1.txt file2.txt > combined.txt` This concatenates the contents of file1.txt and file2.txt, then saves the result to combined.txt.

- `tac` - Display files in reverse (last line first)
- `rev` - Reverse characters in each line
- `sort filename` - Sort lines in a file
- `sort -r filename` - Sort lines in reverse order
- `sort -n filename` - Sort lines numerically
- `sort -k2,2 filename` - Sort lines based on the second field
- `uniq filename` - Remove duplicate adjacent lines
- `uniq -c filename` - Count occurrences of lines
- `uniq -d filename` - Show only duplicate lines
- `comm file1 file2` - Compare sorted files line by line
- `join file1 file2` - Join lines of two files on a common field

Text Manipulation

- `cut -d: -f1 filename` - Extract specific fields from each line

Example: `cut -d, -f1,3 data.csv` This extracts the 1st and 3rd fields from a comma-separated file. The `-d,` specifies comma as the delimiter, and `-f1,3` selects fields 1 and 3.

- `paste file1 file2` - Merge lines of files side by side
- `tr 'a-z' 'A-Z' < filename` - Translate characters (e.g., lowercase to uppercase)
- `tr -d ':' < filename` - Delete specific characters
- `tr -s ' ' < filename` - Squeeze multiple occurrences of characters into one
- `fold -w 80 filename` - Wrap lines to specified width
- `fmt filename` - Simple text formatter
- `pr filename` - Format text for printing
- `column -t filename` - Format input into a table
- `expand filename` - Convert tabs to spaces
- `unexpand filename` - Convert spaces to tabs

Pattern Matching and Regular Expressions

- `grep pattern filename` - Search for a pattern in a file

Example: `grep "error" logfile.txt` This searches for the word "error" in logfile.txt and prints matching lines.

- `grep -i pattern filename` - Case-insensitive search
- `grep -v pattern filename` - Show lines that don't match the pattern
- `grep -n pattern filename` - Show line numbers with matches
- `grep -r pattern directory` - Recursive search in directory
- `grep -e pattern1 -e pattern2 filename` - Search for multiple patterns
- `grep -A3 pattern filename` - Show 3 lines after the match
- `grep -B2 pattern filename` - Show 2 lines before the match
- `grep -C1 pattern filename` - Show 1 line before and after the match
- `egrep pattern filename` - Extended regular expression search (same as `grep -E`)

Stream Editor

- `sed 's/old/new/' filename` - Replace first occurrence of pattern in each line

Example: `sed 's/error/warning/g' logfile.txt` This replaces all occurrences of "error" with "warning" in logfile.txt. The `g` flag makes it replace all occurrences in each line, not just the first one.

- `sed 's/old/new/g' filename` - Replace all occurrences of pattern
- `sed '1,5s/old/new/g' filename` - Replace pattern only in lines 1-5
- `sed '/pattern/d' filename` - Delete lines matching pattern

- `sed '/^$/d' filename` - Delete empty lines
- `sed -i 's/old/new/g' filename` - Edit file in place
- `sed -i.bak 's/old/new/g' filename` - Edit file in place with backup
- `sed -n '5,10p' filename` - Print only lines 5-10
- `sed 'G' filename` - Add empty line after each line

AWK Programming Language

- `awk '{print $1}' filename` - Print first field of each line

Example: `awk '{print $1,$3}' data.txt` This prints the 1st and 3rd fields of each line in data.txt. In AWK, fields are automatically split by whitespace, and accessed as \$1, \$2, etc.

- `awk -F: '{print $1}' filename` - Specify field separator
- `awk '{sum+=$1} END {print sum}' filename` - Sum first field
- `awk 'NR==10 {print}' filename` - Print only line 10
- `awk 'NF > 3' filename` - Print lines with more than 3 fields
- `awk 'length($0) > 80' filename` - Print lines longer than 80 characters
- `awk '/pattern/' filename` - Print lines matching pattern
- `awk 'BEGIN {FS=":"; OFS=","} {print $1,$3}' filename` - Change field separator
- `awk '{gsub(/pattern/, "replacement"); print}' filename` - Replace all occurrences

User & Group Management

User Information

- `who` - Show who is logged in
- `w` - Show who is logged in and what they're doing
- `id` - Display user and group IDs
- `id username` - Display user and group IDs for a specific user
- `groups` - Show groups the current user belongs to
- `groups username` - Show groups a specific user belongs to
- `whoami` - Display current username
- `users` - List users currently logged in
- `last` - Show listing of last logged in users
- `lastlog` - Show the most recent login of all users
- `finger username` - Display information about a user

User Management

- `useradd username` - Create a new user

Example: `useradd -m -s /bin/bash john` This creates a new user named "john" with a home directory (-m) and the bash shell (-s /bin/bash).

- `useradd -m username` - Create a new user with home directory
- `useradd -m -d /custom/home username` - Create user with custom home directory
- `useradd -m -s /bin/bash username` - Create user with specific shell
- `useradd -m -u 1500 username` - Create user with specific UID
- `useradd -m -g groupname username` - Create user with specific primary group
- `useradd -m -G group1,group2 username` - Create user with additional groups
- `adduser username` - Interactive user creation (more user-friendly)
- `usermod -l newname oldname` - Change username
- `usermod -g groupname username` - Change primary group
- `usermod -G group1,group2 username` - Replace all secondary groups
- `usermod -aG groupname username` - Add user to a group
- `usermod -s /bin/bash username` - Change user's shell
- `usermod -L username` - Lock a user account
- `usermod -U username` - Unlock a user account
- `userdel username` - Delete a user
- `userdel -r username` - Delete a user and their home directory
- `passwd username` - Set or change user password
- `passwd -l username` - Lock a user's password
- `passwd -u username` - Unlock a user's password
- `passwd -e username` - Force user to change password at next login
- `chage -l username` - View password expiry information
- `chage -E YYYY-MM-DD username` - Set account expiry date
- `chage -M days username` - Set maximum password age
- `su username` - Switch to another user
- `su -` - Switch to root user with environment
- `sudo command` - Execute command as superuser
- `sudo -i` - Start an interactive shell as superuser

- `sudo -u username command` - Execute command as another user
- `visudo` - Edit the sudoers file safely

Group Management

- `groupadd groupname` - Create a new group
- `groupadd -g 1000 groupname` - Create a group with specific GID
- `groupmod -n newname oldname` - Rename a group
- `groupdel groupname` - Delete a group
- `gpasswd -a username groupname` - Add a user to a group
- `gpasswd -d username groupname` - Remove a user from a group
- `newgrp groupname` - Change current group ID during login session

Process Management

Viewing Processes

- `ps` - Display current user's processes
- `ps aux` - Display all processes with detailed information

Example: `ps aux | grep apache` This shows all processes and filters for ones containing "apache". The `a` shows processes from all users, `u` provides detailed user-oriented output, and `x` includes processes without a controlling terminal.

- `ps -ef` - Display all processes in full format
- `ps -u username` - Display processes for a specific user
- `ps -p PID` - Display information for a specific process ID
- `ps --forest` - Display process tree
- `ps -o pid,user,%cpu,%mem,cmd` - Custom output format
- `pstree` - Display processes in a tree format
- `pgrep processname` - Find process IDs by name
- `pidof processname` - Find process IDs by name (alternative)
- `top` - Dynamic real-time view of processes
- `htop` - Interactive process viewer (more user-friendly than top)
- `atop` - Advanced system & process monitor
- `pgrep pattern` - Find process IDs matching a pattern

Controlling Processes

- `kill PID` - Send termination signal to a process
- `kill -9 PID` - Force kill a process
- `kill -SIGTERM PID` - Send termination signal

- `kill -SIGKILL PID` - Send kill signal
- `kill -SIGHUP PID` - Send hang-up signal (often used to reload configuration)
- `kill -SIGSTOP PID` - Pause a process
- `kill -SIGCONT PID` - Resume a paused process
- `killall processname` - Kill processes by name
- `pkill pattern` - Kill processes matching a pattern
- `xkill` - Kill a process by clicking on its window (X11)
- `renice priority PID` - Change priority of a running process
- `nice -n priority command` - Start a process with adjusted priority
- `nohup command &` - Run command immune to hangups
- `bg` - Resume jobs in the background
- `fg` - Bring jobs to the foreground
- `jobs` - List background jobs
- `Ctrl+Z` - Suspend current foreground process
- `Ctrl+C` - Interrupt/terminate current foreground process
- `timeout seconds command` - Run a command with a time limit
- `watch command` - Execute a command periodically and display output
- `at time` - Schedule a one-time job
- `atq` - List pending at jobs
- `atrm jobid` - Remove a scheduled at job
- `crontab -l` - List scheduled cron jobs
- `crontab -e` - Edit scheduled cron jobs
- `crontab -r` - Remove all crontab jobs

Package Management

APT (Debian/Ubuntu)

- `apt update` - Update package index
- `apt upgrade` - Upgrade installed packages
- `apt full-upgrade` - Upgrade packages with package removal if needed
- `apt install package` - Install a package

Example: `apt install nginx` This installs the nginx web server package.

- `apt remove package` - Remove a package
- `apt purge package` - Remove package and configuration files
- `apt autoremove` - Remove automatically installed dependencies
- `apt search keyword` - Search for packages
- `apt show package` - Show package details
- `apt list --installed` - List installed packages
- `apt list --upgradable` - List upgradable packages

- `apt-cache policy package` - Show package installation candidate
- `apt-cache depends package` - Show package dependencies
- `apt-cache rdepends package` - Show reverse dependencies
- `apt-mark hold package` - Prevent package from being upgraded
- `apt-mark unhold package` - Allow package to be upgraded again
- `dpkg -i package.deb` - Install a local .deb package
- `dpkg -r package` - Remove an installed package
- `dpkg -l` - List installed packages
- `dpkg -L package` - List files installed by a package
- `dpkg -S /path/to/file` - Find which package a file belongs to

YUM/DNF (RHEL/CentOS/Fedora)

- `yum update` - Update package index and upgrade packages
- `yum install package` - Install a package
- `yum remove package` - Remove a package
- `yum search keyword` - Search for packages
- `yum info package` - Show package details
- `yum list installed` - List installed packages
- `yum list available` - List available packages
- `yum provides /path/to/file` - Find which package provides a file
- `yum clean all` - Clean all cache files
- `yum-config-manager --enable repository` - Enable a repository
- `yum-config-manager --disable repository` - Disable a repository
- `rpm -i package.rpm` - Install a local .rpm package
- `rpm -e package` - Remove an installed package
- `rpm -q package` - Query if a package is installed
- `rpm -ql package` - List files installed by a package
- `rpm -qf /path/to/file` - Find which package a file belongs to
- `dnf update` - Update package index and upgrade packages (newer version of yum)
- `dnf install package` - Install a package
- `dnf remove package` - Remove a package
- `dnf search keyword` - Search for packages
- `dnf info package` - Show package details

Pacman (Arch Linux)

- `pacman -Syu` - Synchronize package database and update packages
- `pacman -S package` - Install a package
- `pacman -R package` - Remove a package
- `pacman -Rs package` - Remove a package and its dependencies
- `pacman -Ss keyword` - Search for packages

- `pacman -Qi package` - Display information about an installed package
- `pacman -Ql package` - List files owned by a package
- `pacman -Qo /path/to/file` - Find which package owns a file
- `pacman -Sc` - Clean package cache

Snap (Universal)

- `snap install package` - Install a snap package
- `snap remove package` - Remove a snap package
- `snap refresh package` - Update a snap package
- `snap list` - List installed snap packages
- `snap find keyword` - Search for snap packages
- `snap info package` - Show details about a snap package

Service Management

Systemd

- `systemctl start service` - Start a service

Example: `systemctl start nginx` This starts the nginx web server service.

- `systemctl stop service` - Stop a service
- `systemctl restart service` - Restart a service
- `systemctl reload service` - Reload service configuration
- `systemctl status service` - Check service status
- `systemctl enable service` - Enable service to start at boot
- `systemctl disable service` - Disable service from starting at boot
- `systemctl is-active service` - Check if service is running
- `systemctl is-enabled service` - Check if service starts at boot
- `systemctl mask service` - Completely disable a service
- `systemctl unmask service` - Remove the masking of a service
- `systemctl list-units --type=service` - List all services
- `systemctl list-units --type=service --state=running` - List running services
- `systemctl daemon-reload` - Reload systemd manager configuration
- `systemctl reboot` - Reboot the system
- `systemctl poweroff` - Shut down the system
- `systemctl suspend` - Suspend the system

- `systemctl hibernate` - Hibernate the system
- `systemctl hybrid-sleep` - Hybrid sleep the system

SysVinit (Legacy)

- `service service start` - Start a service
- `service service stop` - Stop a service
- `service service restart` - Restart a service
- `service service status` - Check service status
- `chkconfig service on` - Enable service to start at boot
- `chkconfig service off` - Disable service from starting at boot
- `chkconfig --list` - List all services and their run levels
- `update-rc.d service defaults` - Enable service with default settings
- `update-rc.d -f service remove` - Remove service from startup

Disk & Storage Management

Disk Partitioning

- `fdisk -l` - List disk partitions

Example: `fdisk -l /dev/sda` This lists all partitions on the `/dev/sda` disk.

- `fdisk /dev/sdX` - Start the fdisk partition utility for disk sdX
- `cfdisk /dev/sdX` - Start the cfdisk (curses-based) partition utility
- `parted /dev/sdX` - Start the parted partition utility
- `parted -l` - List disk partitions (alternative to fdisk -l)
- `sfdisk -l /dev/sdX` - Display partition table
- `gdisk /dev/sdX` - GPT fdisk utility for GPT partitions

File Systems

- `mkfs.ext4 /dev/sdXY` - Create an ext4 filesystem on partition
- `mkfs.xfs /dev/sdXY` - Create an XFS filesystem
- `mkfs.btrfs /dev/sdXY` - Create a Btrfs filesystem
- `mkfs.fat -F32 /dev/sdXY` - Create a FAT32 filesystem
- `mkfs.ntfs /dev/sdXY` - Create an NTFS filesystem
- `blkid` - Display block device attributes (UUID, type)
- `blkid /dev/sdXY` - Show UUID of a specific partition
- `fsck /dev/sdXY` - Check and repair a filesystem
- `fsck.ext4 -f /dev/sdXY` - Force check an ext4 filesystem
- `tune2fs -l /dev/sdXY` - Display ext filesystem parameters
- `resize2fs /dev/sdXY` - Resize an ext filesystem

Mounting

- `mount /dev/sdXY /mnt` - Mount a filesystem
- `mount -t type /dev/sdXY /mnt` - Mount a specific filesystem type
- `mount -o options /dev/sdXY /mnt` - Mount with specific options
- `umount /mnt` - Unmount a filesystem
- `umount -l /mnt` - Lazy unmount (unmount when no longer busy)
- `umount -f /mnt` - Force unmount
- `mount -a` - Mount all filesystems in `/etc/fstab`
- `swapon -s` - Display swap usage
- `swapon /dev/sdXY` - Enable a swap partition
- `swapoff /dev/sdXY` - Disable a swap partition
- `lsblk` - List block devices and their mount points
- `findmnt` - List all mounted filesystems

Logical Volume Management (LVM)

- `pvdisplay` - Display physical volumes
- `pvcreate /dev/sdX` - Create a physical volume
- `pvremove /dev/sdX` - Remove a physical volume
- `vgdisplay` - Display volume groups
- `vgcreate vname /dev/sdX` - Create a volume group
- `vgextend vname /dev/sdY` - Add a physical volume to a volume group
- `vgreduce vname /dev/sdY` - Remove a physical volume from a volume group
- `vgremove vname` - Remove a volume group
- `lvdisplay` - Display logical volumes
- `lvcreate -L size -n lvname vname` - Create a logical volume
- `lvresize -L +size /dev/vname/lvname` - Increase logical volume size
- `lvresize -L -size /dev/vname/lvname` - Decrease logical volume size
- `lvremove /dev/vname/lvname` - Remove a logical volume

Disk Usage Analysis

- `df -h` - Show disk space usage in human-readable format
- `df -i` - Show inode usage
- `du -sh /path` - Show directory size in human-readable format
- `du -h --max-depth=1 /path` - Show size of direct subdirectories
- `ncdu /path` - NCurses Disk Usage analyzer (interactive)
- `iotop` - Monitor disk I/O usage by processes
- `iostat` - Report CPU and I/O statistics
- `smartctl -a /dev/sdX` - Display SMART information for a disk
- `badblocks -v /dev/sdX` - Check for bad blocks on a disk
- `hdparm -i /dev/sdX` - Display disk parameters
- `hdparm -tT /dev/sdX` - Perform disk read speed test

System Configuration

Network Configuration

- `ip addr show` - Display network interfaces and IP addresses
- `ip addr add 192.168.1.100/24 dev eth0` - Add IP address to interface

- `ip addr del 192.168.1.100/24 dev eth0` - Remove IP address from interface
- `ip link set eth0 up` - Bring interface up
- `ip link set eth0 down` - Bring interface down
- `ip route show` - Display routing table
- `ip route add default via 192.168.1.1` - Add default gateway
- `ip route del default` - Remove default gateway
- `ifconfig eth0 192.168.1.100 netmask 255.255.255.0` - Configure IP (deprecated)
- `route add default gw 192.168.1.1` - Add default gateway (deprecated)
- `dhclient eth0` - Obtain IP address via DHCP
- `nmcli device status` - Show NetworkManager device status
- `nmcli connection show` - List NetworkManager connections
- `nmcli connection up id "connection_name"` - Activate a connection
- `nmcli device wifi list` - List available Wi-Fi networks
- `nmcli device wifi connect SSID password PASSWORD` - Connect to Wi-Fi
- `netstat -rn` - Display routing table
- `netstat -tulnp` - Show listening ports and associated processes
- `ss -tulnp` - Modern alternative to netstat
- `dig domain` - DNS lookup utility
- `host domain` - DNS lookup utility (simpler)
- `nslookup domain` - DNS lookup utility (interactive)
- `traceroute domain` - Trace route to a host
- `mtr domain` - Matt's Traceroute (interactive traceroute)
- `ping host` - Send ICMP echo requests to a host
- `arp -a` - Display ARP cache
- `arping IP` - Ping a host by ARP requests
- `ethtool eth0` - Display or change Ethernet card settings

System Time

- `date` - Display current date and time
- `date -s "2023-05-20 14:30:00"` - Set system date and time
- `timedatectl` - Control system time and date
- `timedatectl list-timezones` - List available time zones
- `timedatectl set-timezone Europe/London` - Set system time zone
- `timedatectl set-ntp true` - Enable NTP synchronization
- `hwclock --show` - Show hardware clock time
- `hwclock --systohc` - Set hardware clock from system time
- `hwclock --hctosys` - Set system time from hardware clock
- `ntpq -p` - Query NTP servers
- `ntpdate pool.ntp.org` - Synchronize with NTP server

Remote Access & File Transfer

SSH (Secure Shell)

- `ssh username@hostname` - Connect to a remote host via SSH

Example: `ssh john@192.168.1.100` This connects to the host at 192.168.1.100 as user "john".

- `ssh -p 2222 username@hostname` - Connect on a specific port
- `ssh -i key.pem username@hostname` - Connect using a private key
- `ssh -X username@hostname` - Enable X11 forwarding
- `ssh -L 8080:localhost:80 username@hostname` - Set up local port forwarding
- `ssh -R 8080:localhost:80 username@hostname` - Set up remote port forwarding
- `ssh -D 8080 username@hostname` - Set up dynamic port forwarding (SOCKS proxy)
- `ssh-keygen` - Generate SSH key pair
- `ssh-keygen -t rsa -b 4096` - Generate RSA key with 4096 bits
- `ssh-keygen -t ed25519` - Generate Ed25519 key (more secure)
- `ssh-copy-id username@hostname` - Copy SSH key to remote host
- `ssh-add` - Add private key to SSH agent
- `ssh-agent` - Start SSH agent
- `sshfs username@hostname:/remote/path /local/path` - Mount remote filesystem
- `fusermount -u /local/path` - Unmount SSHFS filesystem

SCP (Secure Copy)

- `scp file.txt username@hostname:/remote/path` - Copy file to remote host

Example: `scp data.txt john@192.168.1.100:/home/john/` This copies data.txt to john's home directory on the remote host.

- `scp username@hostname:/remote/file.txt local_file.txt` - Copy from remote host
- `scp -r directory username@hostname:/remote/path` - Copy directory recursively
- `scp -P 2222 file.txt username@hostname:/remote/path` - Use specific port
- `scp -i key.pem file.txt username@hostname:/remote/path` - Use private key
- `scp file1.txt file2.txt username@hostname:/remote/path` - Copy multiple files

SFTP (SSH File Transfer Protocol)

- `sftp username@hostname` - Start SFTP session
- `sftp -P 2222 username@hostname` - Connect on a specific port
- `sftp -i key.pem username@hostname` - Connect using private key
- `get remotefile` - Download file (within SFTP)
- `get -r remotedir` - Download directory recursively (within SFTP)
- `put localfile` - Upload file (within SFTP)
- `put -r localdir` - Upload directory recursively (within SFTP)
- `ls` - List files on remote server (within SFTP)

- `cd remotedir` - Change remote directory (within SFTP)
- `lcd localdir` - Change local directory (within SFTP)
- `pwd` - Print remote working directory (within SFTP)
- `lpwd` - Print local working directory (within SFTP)
- `mkdir remotedir` - Create remote directory (within SFTP)
- `rm remotefile` - Delete remote file (within SFTP)
- `rmdir remotedir` - Delete remote directory (within SFTP)
- `bye` or `exit` - Exit SFTP session

RSYNC (Remote Sync)

- `rsync source destination` - Synchronize files and directories

Example: `rsync -avz /local/data/ user@remote:/backup/` This synchronizes the local data directory to the remote backup directory.

Flags:

- `-a`: archive mode (preserves permissions, ownership, timestamps, etc.)
 - `-v`: verbose output
 - `-z`: compress data during transfer
- `rsync -a source destination` - Archive mode (preserve attributes)
- `rsync -z source destination` - Compress during transfer
- `rsync -v source destination` - Verbose output
- `rsync --delete source destination` - Delete files at destination not in source
- `rsync -P source destination` - Show progress
- `rsync -n source destination` - Dry run (no changes made)
- `rsync -u source destination` - Update (skip newer files at destination)
- `rsync -e "ssh -p 2222" source user@host:dest` - Use specific SSH port

Security & Encryption

Firewall Management

- `ufw status` - Check UFW firewall status
- `ufw enable` - Enable UFW firewall
- `ufw disable` - Disable UFW firewall
- `ufw allow 22/tcp` - Allow SSH connections
- `ufw deny 22/tcp` - Deny SSH connections
- `ufw allow from 192.168.1.0/24 to any port 22` - Allow SSH from specific network
- `ufw delete deny 22/tcp` - Remove a rule
- `ufw reset` - Reset all rules to default
- `iptables -L` - List iptables rules
- `iptables -A INPUT -p tcp --dport 22 -j ACCEPT` - Allow SSH connections

- `iptables -A INPUT -p tcp --dport 80 -j ACCEPT` - Allow HTTP connections
- `iptables -A INPUT -j DROP` - Drop all other incoming traffic
- `iptables-save > rules.v4` - Save iptables rules
- `iptables-restore < rules.v4` - Restore iptables rules
- `firewall-cmd --state` - Check firewall status
- `firewall-cmd --get-default-zone` - Get default zone
- `firewall-cmd --list-all` - List all rules
- `firewall-cmd --permanent --add-service=ssh` - Allow SSH permanently
- `firewall-cmd --reload` - Reload firewall rules

User Authentication

- `passwd` - Change your password
- `passwd username` - Change another user's password (as root)
- `chage -l username` - View password expiry information
- `chage -E YYYY-MM-DD username` - Set account expiry date
- `chage -M days username` - Set maximum password age
- `faillog -a` - Show failed login attempts
- `lastlog` - Show last login time for all users
- `getent passwd username` - Get user entry from passwd database
- `pwck` - Verify integrity of password files
- `pwconv` - Convert to shadow passwords
- `grpck` - Verify integrity of group files
- `grpconv` - Convert to shadow group files
- `pam_tally2` - Display failed login attempts
- `pam_tally2 --reset` - Reset failed login counter

File Encryption

- `gpg -c file` - Encrypt a file with a password

Example: `gpg -c sensitive_data.txt` This encrypts `sensitive_data.txt` with a password, creating `sensitive_data.txt.gpg`.

- `gpg file.gpg` - Decrypt a GPG-encrypted file
- `gpg --gen-key` - Generate a GPG key pair
- `gpg --list-keys` - List GPG keys
- `gpg -e -r recipient file` - Encrypt a file for a recipient
- `gpg --export -a "User Name" > public.key` - Export public key
- `gpg --import public.key` - Import public key
- `openssl enc -aes-256-cbc -in file -out file.enc` - Encrypt file with OpenSSL
- `openssl enc -d -aes-256-cbc -in file.enc -out file` - Decrypt file
- `openssl genrsa -out private.key 2048` - Generate RSA private key
- `openssl rsa -in private.key -pubout -out public.key` - Extract public key

- `ccrypt file` - Encrypt or decrypt a file with ccrypt
- `ccrypt -c file.cpt` - Decrypt a ccrypt-encrypted file
- `7z a -p archive.7z file` - Create password-protected 7zip archive
- `zip -e archive.zip file` - Create password-protected zip archive
- `unzip archive.zip` - Extract zip archive (will prompt for password if needed)

System Security

- `chroot /path/to/new/root command` - Run command with different root directory
- `ulimit -a` - Show all current resource limits
- `ulimit -n 2048` - Set maximum number of open file descriptors
- `systemd-analyze security` - Analyze system security
- `journalctl -f` - Monitor system logs in real-time
- `journalctl -u service` - View logs for a specific service
- `journalctl --since "2023-05-20 14:00:00"` - View logs since a specific time
- `ausearch -m authentication` - Search audit logs for authentication events
- `auditctl -l` - List all audit rules
- `lynis audit system` - Perform security audit with Lynis
- `rkhunter --check` - Check for rootkits
- `chkrootkit` - Check for rootkits (alternative)
- `aide --check` - Check for file integrity with AIDE
- `tripwire --check` - Check for file integrity with Tripwire
- `clamav-daemon` - Start ClamAV daemon
- `clamscan -r /path/to/scan` - Scan directory for viruses
- `freshclam` - Update ClamAV virus definitions

Shell Scripting

Basics

- `#!/bin/bash` - Shebang line for Bash scripts
- `#!/bin/sh` - Shebang line for POSIX shell scripts
- `#!/usr/bin/env python3` - Shebang line for Python scripts
- `#` - Comment - Single-line comment
- `echo "Hello, world!"` - Print text to standard output
- `printf "Hello, %s!\n" "world"` - Formatted printing
- `read variable` - Read user input into a variable
- `read -p "Enter your name: " name` - Prompt for user input
- `read -s password` - Read input without displaying (for passwords)
- `exit 0` - Exit script with success status
- `exit 1` - Exit script with error status
- `set -e` - Exit script if any command fails
- `set -x` - Print each command before executing (debug mode)
- `set -u` - Treat unset variables as an error
- `set -o pipefail` - Fail if any command in a pipe fails

Variables

- `variable="value"` - Assign value to a variable
- `echo $variable` - Access variable value
- `echo ${variable}` - Access variable value (preferred for clarity)
- `readonly variable="value"` - Create a read-only variable
- `unset variable` - Unset (remove) a variable
- `export variable="value"` - Export variable to environment
- `local variable="value"` - Create a local variable in a function
- `variable=${variable:-default}` - Use default value if variable is unset
- `variable=${variable:=default}` - Set default value if variable is unset
- `variable=${variable:+alternate}` - Use alternate value if variable is set
- `variable=${variable:offset:length}` - String slicing
- `${#variable}` - Get length of string variable
- `${variable#pattern}` - Remove shortest match from beginning
- `${variable##pattern}` - Remove longest match from beginning
- `${variable%pattern}` - Remove shortest match from end
- `${variable%%pattern}` - Remove longest match from end
- `${variable/pattern/replacement}` - Replace first occurrence
- `${variable//pattern/replacement}` - Replace all occurrences
- `${variable^}` - Convert first character to uppercase
- `${variable^^}` - Convert all characters to uppercase
- `${variable,}` - Convert first character to lowercase
- `${variable,,}` - Convert all characters to lowercase
- `$0` - Script name
- `$1, $2, ...` - Positional parameters (command-line arguments)
- `$#` - Number of positional parameters
- `$@` - All positional parameters (preserves quoting)
- `$*` - All positional parameters (as a single word)
- `$?` - Exit status of last command
- `$$` - Process ID of current shell
- `$!` - Process ID of last background command

Bash Shell Scripting: Conditional Statements, File Operations, and More

Arithmetic Operators

Bash provides arithmetic operations using the `$ ( ( ) )` syntax. This allows for integer calculations directly within your scripts.

Basic Arithmetic Operators

```
# Addition
echo $((x + y))

# Subtraction
echo $((x - y))

# Multiplication
echo $((x * y))

# Division (integer division)
echo $((x / y))

# Modulus (remainder)
echo $((x % y))

# Exponentiation
echo $((x ** y))
```

Example:

```
#!/bin/bash
# Basic calculator
read -p "Enter first number: " x
read -p "Enter second number: " y

echo "Addition: $((x + y))"
echo "Subtraction: $((x - y))"
echo "Multiplication: $((x * y))"
echo "Division: $((x / y))"
echo "Remainder: $((x % y))"
echo "Exponentiation: $((x ** y))"
```

Compound Assignment Operators

```
# Addition assignment
((x += y)) # Equivalent to x = x + y

# Subtraction assignment
((x -= y)) # Equivalent to x = x - y

# Multiplication assignment
((x *= y)) # Equivalent to x = x * y

# Division assignment
((x /= y)) # Equivalent to x = x / y

# Modulus assignment
((x %= y)) # Equivalent to x = x % y
```

Example:

```
#!/bin/bash
# Compound assignment demo
x=10
echo "Initial value: $x"

((x += 5))
echo "After x += 5: $x"

((x -= 3))
echo "After x -= 3: $x"

((x *= 2))
echo "After x *= 2: $x"

((x /= 4))
echo "After x /= 4: $x"

((x %= 3))
echo "After x %= 3: $x"
```

Increment and Decrement Operators

```
# Pre-increment
((++x)) # Increment x before using its value

# Post-increment
((x++)) # Use x's value, then increment

# Pre-decrement
((--x)) # Decrement x before using its value

# Post-decrement
((x--)) # Use x's value, then decrement
```

Example:

```
#!/bin/bash
# Increment/decrement demo
x=5
echo "Initial value: $x"

y=$((x++))
echo "y=$((x++)): y=$y, x=$x"

x=5
y=$((++x))
echo "y=$((++x)): y=$y, x=$x"

x=5
y=$((x--))
echo "y=$((x--)): y=$y, x=$x"

x=5
y=$((--x))
echo "y=$((--x)): y=$y, x=$x"
```

Bitwise Operators


```
# Bitwise AND
echo $((x & y))

# Bitwise OR
echo $((x | y))

# Bitwise XOR
echo $((x ^ y))

# Bitwise NOT
echo $((~x))

# Left shift
echo $((x << y))

# Right shift
echo $((x >> y))
```

Example:

```
#!/bin/bash
# Bitwise operations demo
x=5 # Binary: 101
y=3 # Binary: 011

echo "Bitwise AND (5 & 3): $((x & y))" # Result: 1 (001)
echo "Bitwise OR (5 | 3): $((x | y))" # Result: 7 (111)
echo "Bitwise XOR (5 ^ 3): $((x ^ y))" # Result: 6 (110)
echo "Bitwise NOT (~5): $((~x))" # Result: -6 (depends on word size)
echo "Left shift (5 << 1): $((x << 1))" # Result: 10 (1010)
echo "Right shift (5 >> 1): $((x >> 1))" # Result: 2 (10)
```

The Difference Between Bracket Types in Bash

Bash uses several types of brackets with different purposes:

Single Brackets []

Single brackets [] are a more portable form for conditional expressions but have limitations:

- Less feature-rich than double brackets
- Require quoting variables to prevent word splitting
- Require escape characters for certain operators
- Command substitution and pathname expansion occur

Example:

```
#!/bin/bash

# Single bracket conditionals
file="example.txt"
string="hello world"

# File test
if [ -f "$file" ]; then
    echo "File exists"
fi

# String comparison (requires quoting)
if [ "$string" = "hello world" ]; then
    echo "Strings match"
fi

# Numeric comparison
if [ "$count" -eq 5 ]; then
    echo "Count equals 5"
fi

# AND condition (requires -a or && between brackets)
if [ -f "$file" ] && [ "$count" -eq 5 ]; then
    echo "Both conditions are true"
fi
```

Double Brackets [[]]

Double brackets `[[ ]]` are a Bash-specific extension that provides more features:

- Pattern matching and regex support
- No need for quoting variables (though still recommended)
- More intuitive syntax for logical operators
- Word splitting and pathname expansion do not occur
- Generally safer to use

Example:

```
#!/bin/bash

# Double bracket conditionals
file="example.txt"
string="hello world"
pattern="h*d"

# File test
if [[ -f $file ]]; then
    echo "File exists"
fi

# String comparison (quoting still recommended but not required)
if [[ $string == "hello world" ]]; then
    echo "Strings match"
fi

# Pattern matching
if [[ $string == $pattern ]]; then
    echo "String matches pattern"
fi

# Regular expression matching
if [[ $string =~ ^h.*d$ ]]; then
    echo "String matches regex"
fi

# AND condition (cleaner syntax)
if [[ -f $file && $count -eq 5 ]]; then
    echo "Both conditions are true"
fi
```

Double Parentheses (())

Double parentheses (()) are used specifically for arithmetic operations and evaluations:

- Allows C-like arithmetic syntax
- Variables don't need \$ prefix inside
- Returns 0 (true) if result is non-zero, 1 (false) if result is zero
- Supports all arithmetic operators

Example:

```
#!/bin/bash

# Double parentheses for arithmetic
x=5
y=10

# Arithmetic evaluation
if ((x < y)); then
    echo "x is less than y"
fi

# Calculation with assignment
((result = x * y))
echo "x * y = $result"

# Increment in place
((x++))
echo "After increment, x = $x"

# Complex arithmetic expression
((result = x * 2 + y / 2))
echo "Result of expression: $result"

# Bitwise operations
((bits = (1 << 3) | 2))
echo "Bitwise result: $bits"
```

Curly Braces

Curly braces serve multiple purposes in Bash:

- **Parameter expansion:** \${variable}
- **Command grouping:** { commands; }
- **Brace expansion:** {1..5} or {a,b,c}

Example:

```
#!/bin/bash

# Curly brace usage
name="John"

# Parameter expansion with manipulation
echo "${name}"
echo "${name}s"           # Append directly
echo "${name^}"          # First letter uppercase
echo "${name^^}"         # All uppercase
echo "${name:0:2}"       # First two characters
echo "${name/o/a}"       # Replace first 'o' with 'a'

# Command grouping (executes in current shell)
{ echo "These commands"; echo "Run in sequence"; echo "In current shell"; }

# Brace expansion for sequences
echo {1..5}              # Outputs: 1 2 3 4 5
echo {a..e}              # Outputs: a b c d e
echo {01..05}            # Outputs: 01 02 03 04 05
echo {a,b,c}_file        # Outputs: a_file b_file c_file
```

Arithmetic Comparison Operators

When used with `(( ))` or within arithmetic expressions, these operators work like in C:

```
# Equal to
((x == y))

# Not equal to
((x != y))

# Less than
((x < y))

# Less than or equal to
((x <= y))

# Greater than
((x > y))

# Greater than or equal to
((x >= y))

# Logical AND
((x < 10 && y > 5))

# Logical OR
((x < 3 || y > 15))

# Logical NOT
(!(x))
```

Example:

```
#!/bin/bash

# Arithmetic comparisons
x=7
y=10

if ((x < y)); then
    echo "x is less than y"
fi

if ((x * 2 > y)); then
    echo "x times 2 is greater than y"
fi

# Ternary operator (condition ? value_if_true : value_if_false)
max=$((x > y ? x : y))
echo "Maximum value is: $max"

# Using logical operators
if ((x > 5 && y < 20)); then
    echo "Both conditions are true"
fi

# Check if a number is even
if ((x % 2 == 0)); then
    echo "x is even"
else
    echo "x is odd"
fi
```

Practical Examples Combining These Concepts

Example 1: Finding Prime Numbers

```
#!/bin/bash
# Check if a number is prime
read -p "Enter a number to check if it's prime: " num

if ((num < 2)); then
    echo "$num is not a prime number"
    exit 0
fi

is_prime=1
for ((i=2; i*i<=num; i++)); do
    if ((num % i == 0)); then
        is_prime=0
        break
    fi
done

if ((is_prime)); then
    echo "$num is a prime number"
else
    echo "$num is not a prime number"
fi
```

Example 2: Simple Calculator Script


```
#!/bin/bash

# Interactive calculator script

echo "Simple Calculator"
echo "-----"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
echo "5. Modulus"
echo "6. Exponentiation"
read -p "Choose operation (1-6): " choice

read -p "Enter first number: " a
read -p "Enter second number: " b

case $choice in
    1)
        result=$((a + b))
        op="+"
        ;;
    2)
        result=$((a - b))
        op="-"
        ;;
    3)
        result=$((a * b))
        op="*"
        ;;
    4)
        if ((b == 0)); then
            echo "Error: Division by zero"
            exit 1
        fi
        result=$((a / b))
        op="/"
        ;;
    5)
        if ((b == 0)); then
            echo "Error: Division by zero"
            exit 1
        fi
        result=$((a % b))
        op="%"
        ;;
    6)
        result=$((a ** b))
        op="^"
        ;;
```

```
        *)
        echo "Invalid choice"
        exit 1
        ;;
    esac

    echo "Result: $a $op $b = $result"
```

Example 3: Combining Different Bracket Types

```

#!/bin/bash

# Script demonstrating different bracket types

file="data.txt"
count=5
threshold=10
values=(8 15 3 42 7)

# Create test file if it doesn't exist
if [[ ! -f $file ]]; then
    echo "Creating test file $file"
    for ((i=0; i<5; i++)); do
        echo "Line $i: Value=${values[i]}" >> $file
    done
fi

# Process file if it exists and is readable
if [[ -f $file && -r $file ]]; then
    echo "Processing $file..."

    # Calculate sum of values
    sum=0
    for value in "${values[@]"; do
        ((sum += value))
    done

    echo "Sum of values: $sum"
    average=$((sum / ${#values[@]}))
    echo "Average: $average"

    # Check if average exceeds threshold
    if ((average > threshold)); then
        echo "Average exceeds threshold ($threshold)"

        # Find values above average
        echo "Values above average:"
        for value in "${values[@]"; do
            if ((value > average)); then
                echo "- $value"
            fi
        done
    else
        echo "Average is below or equal to threshold"
    fi

    # Pattern matching with double brackets
    grep_count=$(grep -c "Value=[0-9]" $file)
    if [[ $grep_count -gt 0 ]]; then
        echo "Found $grep_count lines with single-digit values"
    fi
fi

```

```
    fi
else
    echo "Cannot access $file"
fi
```

Conditional Statements

Basic If-Else Statement

```
if [ condition ]; then
    commands
else
    commands
fi
```

Example:

```
#!/bin/bash
# Check if user is root
if [ "$USER" = "root" ]; then
    echo "You are running as root"
else
    echo "You are running as $USER"
fi
```

If-Elif-Else Statement

```
if [ condition1 ]; then
    commands
elif [ condition2 ]; then
    commands
else
    commands
fi
```

Example:

```
#!/bin/bash
# Check file size and categorize it
filesize=$(du -k "$1" | cut -f1)

if [ $filesize -lt 1024 ]; then
    echo "File is small (less than 1MB)"
elif [ $filesize -lt 10240 ]; then
    echo "File is medium (between 1MB and 10MB)"
else
    echo "File is large (over 10MB)"
fi
```

File and Directory Tests

Check if File Exists

```
[ -e file ]
```

Example:

```
#!/bin/bash
# Check if config file exists
if [ -e "/etc/nginx/nginx.conf" ]; then
    echo "Nginx configuration file exists"
else
    echo "Nginx configuration file not found"
fi
```

Check if File Exists and is a Regular File

```
[ -f file ]
```

Example:

```
#!/bin/bash
# Check if a file is a regular file
if [ -f "$1" ]; then
    echo "$1 is a regular file"
else
    echo "$1 is not a regular file or does not exist"
fi
```

Check if Directory Exists

```
[ -d directory ]
```

Example:

```
#!/bin/bash
# Check if logs directory exists, create if not
if [ -d "/var/log/myapp" ]; then
    echo "Log directory exists"
else
    echo "Creating log directory..."
    mkdir -p /var/log/myapp
fi
```

Check if File is Readable

```
[ -r file ]
```

Example:

```
#!/bin/bash
# Verify a file is readable before processing
if [ -r "data.csv" ]; then
    echo "Processing data file..."
    cat data.csv | wc -l
else
    echo "Error: Cannot read data.csv"
    exit 1
fi
```

Check if File is Writable

```
[ -w file ]
```

Example:

```
#!/bin/bash
# Check if log file is writable
if [ -w "/var/log/myapp.log" ]; then
    echo "Log file is writable, appending data..."
    echo "$(date): Script executed" >> /var/log/myapp.log
else
    echo "Warning: Cannot write to log file"
fi
```

Check if File is Executable

```
[ -x file ]
```

Example:

```
#!/bin/bash
# Check if a script is executable
if [ -x "./deploy.sh" ]; then
    echo "Running deployment script..."
    ./deploy.sh
else
    echo "Error: deploy.sh is not executable"
    echo "Try: chmod +x deploy.sh"
fi
```

Check if File Exists and is Not Empty

```
[ -s file ]
```

Example:

```
#!/bin/bash
# Check if configuration file has content
if [ -s "config.json" ]; then
    echo "Config file has content, proceeding..."
    cp config.json /etc/myapp/
else
    echo "Error: Config file is empty or missing"
    exit 1
fi
```

String Comparisons

Check if Strings are Equal

```
[ string1 = string2 ]
```

Example:

```
#!/bin/bash
# Check password match
if [ "$password" = "$confirm_password" ]; then
    echo "Passwords match, proceeding..."
else
    echo "Error: Passwords do not match"
fi
```

Check if Strings are Not Equal

```
[ string1 != string2 ]
```

Example:

```
#!/bin/bash
# Verify environment is not production
if [ "$ENV" != "production" ]; then
    echo "Debug mode enabled (non-production environment)"
    set -x # Enable debug output
else
    echo "Production mode - debug disabled"
fi
```

Check if String is Empty

```
[ -z string ]
```

Example:

```
#!/bin/bash
# Validate required input
if [ -z "$username" ]; then
    echo "Error: Username cannot be empty"
    exit 1
else
    echo "Creating user $username..."
fi
```

Check if String is Not Empty

```
[ -n string ]
```

Example:


```
#!/bin/bash
# Use a default value if variable is empty
if [ -n "$custom_port" ]; then
    port="$custom_port"
    echo "Using custom port: $port"
else
    port="8080"
    echo "Using default port: $port"
fi
```

Numeric Comparisons

Check if Numbers are Equal

```
[ number1 -eq number2 ]
```

Example:

```
#!/bin/bash
# Check if user has correct response
answer=42
if [ "$user_answer" -eq $answer ]; then
    echo "Correct! The answer to life, the universe, and everything!"
else
    echo "Sorry, that's incorrect."
fi
```

Check if Numbers are Not Equal

```
[ number1 -ne number2 ]
```

Example:

```
#!/bin/bash
# Check if exit code indicates error
if [ $? -ne 0 ]; then
    echo "Previous command failed with exit code $?"
    exit 1
else
    echo "Previous command completed successfully"
fi
```

Check if Number1 is Less Than Number2

```
[ number1 -lt number2 ]
```

Example:

```
#!/bin/bash
# Check if disk space is critically low
available=$(df -k / | awk 'NR==2 {print $4}')
if [ $available -lt 1048576 ]; then # Less than 1GB
    echo "Warning: Low disk space! Only $((($available/1024))MB available"
else
    echo "Disk space is sufficient: $((($available/1024))MB available"
fi
```

Check if Number1 is Less Than or Equal to Number2

```
[ number1 -le number2 ]
```

Example:

```
#!/bin/bash
# Check if CPU load is at or over threshold
load=$(uptime | awk '{print $(NF-2)}' | tr -d ',')
if [ $(echo "$load >= 4.0" | bc) -eq 1 ]; then
    echo "High CPU load detected: $load"
    echo "Consider scaling up resources"
fi
```

Check if Number1 is Greater Than Number2

```
[ number1 -gt number2 ]
```

Example:

```
#!/bin/bash
# Check if memory usage is too high
mem_used_percent=$(free | grep Mem | awk '{print $3/$2 * 100.0}')
if [ $(echo "$mem_used_percent > 90" | bc) -eq 1 ]; then
    echo "Critical: Memory usage at ${mem_used_percent}%"
    echo "Clearing cache..."
    sync; echo 3 > /proc/sys/vm/drop_caches
fi
```

Check if Number1 is Greater Than or Equal to Number2

```
[ number1 -ge number2 ]
```

Example:

```
#!/bin/bash
# Check if enough arguments were provided
if [ $# -ge 3 ]; then
    echo "Processing with source=$1, destination=$2, and mode=$3"
else
    echo "Error: Not enough arguments"
    echo "Usage: $0 <source> <destination> <mode>"
    exit 1
fi
```

Logical Operators

Logical AND

```
[ condition1 ] && [ condition2 ]
```

Example:

```
#!/bin/bash
# Check if file exists and is executable
if [ -f "./backup.sh" ] && [ -x "./backup.sh" ]; then
    echo "Backup script found and is executable"
    ./backup.sh
else
    echo "Backup script missing or not executable"
fi
```

Logical OR

```
[ condition1 ] || [ condition2 ]
```

Example:

```
#!/bin/bash
# Try to use either primary or backup config
if [ -f "/etc/app/config.json" ] || [ -f "/etc/app/config.backup.json" ]; then
    echo "At least one configuration file found"
else
    echo "Error: No configuration files found"
    exit 1
fi
```

Logical NOT

```
[ ! condition ]
```

Example:

```
#!/bin/bash
# Ensure directory doesn't exist before creating
if [ ! -d "./build" ]; then
    echo "Creating build directory..."
    mkdir ./build
else
    echo "Build directory already exists"
    echo "Cleaning previous build..."
    rm -rf ./build/*
fi
```

Regular Expression Matching

```
[[ string =~ regex ]]
```

Example:

```
#!/bin/bash
# Validate email format
email="user@example.com"
if [[ $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]; then
    echo "Valid email format"
else
    echo "Invalid email format"
fi
```

Loops

For Loop

```
for variable in list; do
    commands
done
```

Example:

```
#!/bin/bash
# Process multiple files
for file in *.log; do
    echo "Processing $file..."
    grep "ERROR" "$file" >> error_summary.txt
    echo "Completed processing $file"
done
echo "All log files processed"
```

C-style For Loop

```
for ((i=0; i<10; i++)); do
    commands
done
```

Example:

```
#!/bin/bash
# Generate numbered backup files
echo "Creating 5 backup files..."
for ((i=1; i<=5; i++)); do
    cp data.db "data_backup_${i}.db"
    echo "Created backup $i of 5"
done
echo "Backup complete"
```

While Loop

```
while [ condition ]; do
    commands
done
```

Example:

```
#!/bin/bash

# Retry connection until successful or max attempts reached
attempts=0
max_attempts=5
connected=false

while [ $attempts -lt $max_attempts ] && [ "$connected" = false ]; do
    echo "Attempt $((attempts+1)) of $max_attempts: Connecting to server..."
    if ping -c 1 server.example.com > /dev/null 2>&1; then
        echo "Connection successful!"
        connected=true
    else
        echo "Connection failed. Retrying in 5 seconds..."
        sleep 5
        attempts=$((attempts+1))
    fi
done

if [ "$connected" = false ]; then
    echo "Failed to connect after $max_attempts attempts"
    exit 1
fi
```

Until Loop

```
until [ condition ]; do
    commands
done
```

Example:

```
#!/bin/bash

# Wait until service is available
echo "Waiting for database to become available..."
until [ "$(curl -s http://localhost:5432)" = "ready" ]; do
    echo "Database not ready yet, waiting..."
    sleep 2
done
echo "Database is now available!"
```

Break and Continue

```
# Break example
for file in *; do
    if [ -d "$file" ]; then
        echo "Found directory: $file"
        break # Exit loop on first directory
    fi
done

# Continue example
for num in {1..10}; do
    if [ $(( $num % 2 )) -eq 0 ]; then
        continue # Skip even numbers
    fi
    echo "Processing odd number: $num"
done
```

Case Statement

```
case variable in
    pattern1)
        commands;;
    pattern2)
        commands;;
    *)
        commands;; # Default case
esac
```

Example:

```
#!/bin/bash
# Process different file types
for file in *; do
    case "$file" in
        *.jpg|*.png|*.gif)
            echo "Processing image: $file"
            ./image_optimizer.sh "$file"
            ;;
        *.mp4|*.avi|*.mov)
            echo "Processing video: $file"
            ./video_converter.sh "$file"
            ;;
        *.txt|*.md|*.doc)
            echo "Processing document: $file"
            ./doc_analyzer.sh "$file"
            ;;
        *)
            echo "Unknown file type: $file"
            ;;
    esac
done
```

Select Statement

```
select option in list; do
    commands
done
```

Example:


```
#!/bin/bash

# Interactive menu for system tasks
echo "System Maintenance Menu:"
select task in "Update System" "Clean Temp Files" "Backup Home" "Exit"; do
    case $task in
        "Update System")
            echo "Updating system packages..."
            apt update && apt upgrade -y
            ;;
        "Clean Temp Files")
            echo "Cleaning temporary files..."
            rm -rf /tmp/* /var/tmp/*
            ;;
        "Backup Home")
            echo "Backing up home directory..."
            tar -czf home_backup_$(date +%F).tar.gz /home
            ;;
        "Exit")
            echo "Exiting..."
            break
            ;;
        *)
            echo "Invalid option"
            ;;
    esac
done
echo "Task completed. Select another option or Exit."
```

Functions

Defining and Calling Functions

```
# Method 1
function_name() {
    commands
}

# Method 2
function function_name {
    commands
}

# Calling a function
function_name

# Calling with arguments
function_name arg1 arg2
```

Example:

```

#!/bin/bash

# Define a function to check system health
check_system() {
    echo "Checking system health..."
    echo "Memory usage:"
    free -h
    echo "Disk usage:"
    df -h
    echo "System load:"
    uptime
}

# Define a function with parameters
backup_directory() {
    local source_dir="$1"
    local backup_name="$2"

    if [ -z "$source_dir" ] || [ -z "$backup_name" ]; then
        echo "Error: Missing parameters"
        echo "Usage: backup_directory <source_dir> <backup_name>"
        return 1
    fi

    if [ ! -d "$source_dir" ]; then
        echo "Error: Source directory '$source_dir' not found"
        return 2
    fi

    echo "Backing up $source_dir to $backup_name.tar.gz"
    tar -czf "$backup_name.tar.gz" "$source_dir"
    return $?
}

# Function with return value
is_root() {
    if [ "$EUID" -eq 0 ]; then
        return 0 # true
    else
        return 1 # false
    fi
}

# Main script
echo "Starting system maintenance script"

# Call the system check function
check_system

# Check if running as root

```

```
if is_root; then
    echo "Running with root privileges"
else
    echo "Warning: Not running as root, some operations may fail"
fi

# Backup the etc directory
backup_directory "/etc" "etc_backup_$(date +%F)"
if [ $? -eq 0 ]; then
    echo "Backup completed successfully"
else
    echo "Backup failed with exit code $?"
fi

echo "Maintenance script completed"
```

Arrays

Array Operations

```
# Define an array
array=(item1 item2 item3)

# Assign value to array element
array[0]="item1"

# Access array element
echo ${array[0]}

# Access all array elements
echo ${array[@]}

# Access all array elements as a single word
echo ${array[*]}

# Get array length
echo ${#array[@]}

# Get array indices
echo ${!array[@]}

# Remove array element
unset array[1]

# Remove entire array
unset array

# Append to array
array+=(item4 item5)

# Iterate over array
for item in "${array[@]}; do
    echo "$item"
done
```

Example:

```
#!/bin/bash

# Server management script using arrays

# Define array of servers
servers=("web1.example.com" "web2.example.com" "db1.example.com" "cache.example.com")

echo "Managing ${#servers[@]} servers"

# Add a new server
servers+=("monitor.example.com")
echo "Added new server, now managing ${#servers[@]} servers"

# Show all servers with their index
for i in "${!servers[@]}"; do
    echo "Server $i: ${servers[$i]}"
done

# Perform operation on specific server types
for server in "${servers[@]}"; do
    if [[ $server == web* ]]; then
        echo "Running web server check on $server..."
        # ssh $server "systemctl status nginx"
    elif [[ $server == db* ]]; then
        echo "Running database check on $server..."
        # ssh $server "systemctl status mysql"
    else
        echo "Running general check on $server..."
        # ssh $server "uptime"
    fi
done

# Remove a server from monitoring
echo "Removing server at index 2 (${servers[2]})"
unset servers[2]

# Show remaining servers (note the gap in indices)
echo "Remaining servers:"
for i in "${!servers[@]}"; do
    echo "Server $i: ${servers[$i]}"
done
```

Input/Output Redirection

```
# Redirect stdout to file (overwrite)
command > file

# Redirect stdout to file (append)
command >> file

# Redirect stderr to file
command 2> file

# Redirect both stdout and stderr to file
command &> file

# Redirect file to stdin
command < file

# Pipe stdout of command1 to stdin of command2
command1 | command2

# Pipe both stdout and stderr of command1 to command2
command1 |& command2

# Run command2 only if command1 succeeds
command1 && command2

# Run command2 only if command1 fails
command1 || command2

# Run command in the background
command &

# Group commands in a subshell
(commands)

# Group commands without a subshell
{ commands; }

# Read from stdin and write to both stdout and file
command | tee file

# Create a named pipe
mkfifo pipe

# Open file descriptor for writing
exec 3> file

# Open file descriptor for reading
exec 3< file
```

```
# Close file descriptor
exec 3>&-
```

Example:

```
#!/bin/bash
# Example script demonstrating I/O redirection

echo "Starting log file processing..." > process.log
echo "Processing started at $(date)" >> process.log

# Redirect errors to error log
find /var/log -type f -name "*.log" 2> errors.log

# Save both output and errors
find /etc -name "*.conf" &> all_output.log

# Process input from a file
sort < unsorted.txt > sorted.txt

# Pipe data through multiple commands
cat access.log | grep "ERROR" | wc -l > error_count.txt

# Using tee to see output and save to file
echo "System information:" | tee system_info.log
uname -a | tee -a system_info.log
free -h | tee -a system_info.log

# Command chaining with && and ||
ping -c 1 google.com > /dev/null && echo "Internet is working" || echo "Internet is down"

# Using subshells
echo "Current directory: $(pwd)"

# Using command groups
{ echo "Starting backup"; tar -czf backup.tar.gz /home; echo "Backup complete"; } > backup.log 2>&1

# Custom file descriptors
exec 3> custom.log
echo "This goes to custom.log" >&3
echo "This goes to standard output"
exec 3>&- # Close the file descriptor

echo "Script completed"
```